

Table of Contents

[PatternCompilerDefTopic](#)

[pcpatinfutility.4.1](#)

[pcpatinfutility.4.2](#)

[pcpatinfutility.4.3](#)

[pcpatterncompiler.1.2](#)

[pcpatterncompiler.1.3](#)

[pcpatterncompiler.1.4](#)

[pcpatterncompiler.1.5](#)

[pcpatterncompiler.1.6](#)

[pcpatterncompiler.1.7](#)

[pcpatterncompiler.1.8](#)

[pcpatterncompiler.1.9](#)

[pcpatternlock](#)

[pcpatternreverse.2.01](#)

[pcpatternreverse.2.02](#)

[pcpatternreverse.2.03](#)

[pcpatternreverse.2.04](#)

[pcpatternreverse.2.05](#)

[pcpatternreverse.2.06](#)

[pcpatternreverse.2.07](#)

[pcpatternreverse.2.08](#)

[pcpatternreverse.2.09](#)

[pcpatternreverse.2.10](#)

PatternCompilerDefTopic

<

>

Pattern Compiler

Pattern Compiler

The Pattern Compiler takes an ASCII pattern file that meets the pattern syntax requirements and produces a pattern binary file.

The following topics are available:

- [Pattern Compiler Overview](#)
- [Pattern Compiler Window](#)
- [Pattern Compiler Command-Line Interface](#)

IG-XL also includes:

- [Pattern Reverse Compiler](#), which reads a binary pattern file and recreates the ASCII pattern source file. See [Pattern Reverse Compiler](#).
- [A pattern locking feature](#) that lets you protect a compiled pattern file. See [Pattern Locking](#).
- [A pattern information utility](#), which reads a binary pattern file and prints its contents in ASCII file equivalents. See [Pattern Information Utility](#).

<

>

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

pcpatinfoutility.4.1

<

>

Pattern Information Utility

Pattern Information Utility

The [pattern information command-line interface utility](#) (patinfo) reads a compiled binary pattern (.pat) file of any hardware type and displays its contents in ASCII output equivalents. You can use this utility to automate and script pattern analysis tasks that would take much longer if you opened patterns individually in PatternTool or reverse compiled files. The list below describes some of the many uses for this utility:

- [Retrieve pattern header information so that when you need to reverse compile and edit a pattern, you have the required information on how it was originally compiled so you can recompile it.](#)
- [Calculate how much pattern memory \(VM and SRM\) a set of binary patterns require without loading them on a test system. This is useful for patterns compiled before the introduction of Pattern Control Display on UltraFLEX.](#)
- [Audit patterns for halt opcodes that affect how patterns are executed when bursting a pattern list.](#)
- [Determine the number of pins in a pattern and ensure that those pins are within the designated time domain.](#)
- [Retrieve the time domain, time sets \(tsets\), pin names and types, and labels used in a pattern.](#)
- [Count the number of vectors used in a pattern.](#)

- Count the number of pattern modules and retrieve their names.
- Determine the presence of scan vectors and retrieve the scan pins, pin types, and mode used in a pattern.
- View automatic test pattern generation (ATPG) pattern setup information.

This section describes this utility’s command-line interface and includes the following topics:

- Pattern Information Utility Command-Line Interface
- Pattern Analysis Script Example

pcpatinfoutility.4.2



Pattern Information Utility : Pattern Information Utility Command-Line Interface

Pattern Information Utility Command-Line Interface

The patinfo utility runs from the command prompt and provides various arguments to specify what information it should output.

To use the patinfo utility, open a command prompt window by selecting Start>All Programs>Accessories>Command Prompt.

The command-line interface for this utility is:

patinfo binary pattern file name(s) [-module name] [switches]

Binary Pattern File Names

A binary pattern file name is required, and the expected file extension is .pat. You cannot use ASCII test pattern (.atp) file names. You can specify more than one binary pattern file, and the utility lists output for each file in the order in which they were listed. You cannot use standard wildcard characters * or ? when specifying pattern file names.

When you specify only a pattern without including any switches, the utility always returns basic information about a pattern, including time stamp, system name, and user account information, as shown in the example below:

Magic: 1126293165

Version: 2

Hardware: 0

HasErrors: 0

HW Header Size: 246

Creation time: Tue Nov 28 07:45:03 2006

System: CIRCE

User: Poseidon

Created by: Pattern Compiler

Lock checksum: 0

Orig. Output: C:\Digital_6.20_LabFiles\patterns\func1_SRM.PAT

HSDM-specific version: 9

Pattern File: 'func1_SRM.PAT'

Number of pattern vectors: 32

Number of modules: 1

Module names: func1_srm,

Loading requires: 32 vectors of SRM

0 vectors of vm

Use the -nots switch if you do not want to include the time stamp, system name, and user account information.

Module Names

You can include an optional pattern module name so that the utility only returns information on that specified module. Use this syntax to filter results pertaining to a specific module name in a pattern file:

patinfo pattern file name(s) -module name switches

Note that the following switches return values on a per-module basis:

-symtab	Returns labels and references to labels.
-comtab	Returns comments.
-vectors	Returns VM and SRM vector data.
-ucode	Returns VM and SRM microcode data.
-isl	Returns Instrument Sync Link data.

If you do not include one or more of these switches, then the -module option does not return any specific module-based information.

The utility returns a "module not found" error message if you do not include a module name or the module name is not found in the pattern file. Module names are always listed in the basic information for a pattern.

For example, the command below returns vector information for the first eight vectors starting at vector 0 for the ControllerDrive module:

patinfo digital_pat.pat -module ControllerDrive -vectors 0 8

MODULE INFO

Type: VM
Name: ControllerDrive
Version: 201
Number of Virtual Vectors: 10064
Number of SRM Chan Locs: 0
Number of VM Chan Locs: 10064
HasErrors: 0

VM Data in VM Hardware

0	0x0108	0x0fff	
1	0x0fc8	0x2000	0x2103 0x0fff
2	0x0fc8	0x3040	0x0fff
3	0x0fc8	0x1001	0x0fff
4	0x0fc8	0x1000	0x0fff
5	0x0fc8	0x1001	0x0fff
6	0x0fc8	0x1000	0x0fff
7	0x0fc8	0x1001	0x0fff

Switches

The table below lists the patinfo utility switches. You can prefix a switch word with "/" instead of "-". The two are equivalent. These switches output specific information about the pattern file.

-help	Returns a summary of the patinfo utility syntax and what switches are valid.
-hdr	Returns the common file header. For example: PATTERN FILE HEADER ----- Version: 106 Time domain: Number of Modules: 1 Number of Virtual Vectors: 32 Number of SVM Chan Locs: 32 Number of LVM Chan Locs: 0
-config	Returns the pins or channels and instruments used in pattern. Note that you must use this switch to determine if the pattern contains scan pins. For example: PATTERN CONFIGURATION -----

Field Description Table:

Radix 4 for 1 - pin(s):

oe(1 pin(s)) at offsets: 0 Pin Names: oe
IsGenerated: false
IsDifferential: false Modifier:
HasPerPinPinSetup: false
PinSetupIndex: 0

Radix 4 for 1 - pin(s):

dir(1 pin(s)) at offsets: 4 Pin Names: dir
IsGenerated: false
IsDifferential: false Modifier:
HasPerPinPinSetup: false
PinSetupIndex: 1

Instrument Description Table:

Name: DCVS, 31 bit(s), starting bit 0:

Instrument Bitfield: 0
vcc(1 pin(s)) at offsets: 0 Pin Names: vcc

Pin Setup Table:

oe Pin Names: oe
 Pin setup: NORMAL IsGenerated: true Platform pin setup ID: 1
dir Pin Names: dir
 Pin setup: NORMAL IsGenerated: true Platform pin setup ID: 1

-tset

Returns a list of time set names saved in the pattern. For example:

Tset Name Table:

time_func 0

-symtab

Returns tables of labels and references to labels. This switch returns information on a per-module basis. For example:

MODULE INFO

Type: SRM

Name: func1_srm
 Version: 106
 Number of Virtual Vectors: 32
 Number of SRM Chan Locs: 32
 Number of VM Chan Locs: 0
 HasErrors: 0

Global Symbol Table:

srm_func1: type start, vector 0, SRM offset 0, VM offset -1,
 SCAN offset 0, in SRM

Local Symbol Table:

func_start_srm: type local, vector 0, SRM offset 0, VM offset -1,
 SCAN offset 0, in SRM

loop_it_alt: type local, vector 45, SRM offset 45, VM offset -1,
 SCAN offset 0, in SRM

loop_it_main: type local, vector 14, SRM offset 14, VM offset -1,
 SCAN offset 0, in SRM

strobe_altVS: type local, vector 40, SRM offset 40, VM offset -1,
 SCAN offset 0, in SRM

strobe_mainVS: type local, vector 5, SRM offset 5, VM offset -1,
 SCAN offset 0, in SRM

Resolved Reference Table:

loop_it_alt: type resolved, vector 64, offset 64 in SRM

loop_it_main: type resolved, vector 33, offset 33 in SRM

Unresolved Reference Table:

example_keepalive: type subr, vector 31, offset 31 in SRM

-comtab

Returns comments. This switch returns information on a per-module basis. For example:

MODULE INFO

Type: SRM

Name: func1_srm
 Version: 106
 Number of Virtual Vectors: 32
 Number of SRM Chan Locs: 32
 Number of VM Chan Locs: 0
 HasErrors: 0

Comment Table:

Loop the pattern using the alternate DAC.

Loop the pattern using the main DAC.

- all Returns all parts of the pattern file for all vectors.
- nots Specifies that no time stamp, system name, or user account information is to be printed.
- vectors n m Returns m vectors of VM/SRM data starting from vector n. This switch returns information on a per-module basis. For example, the following command lists eight vectors starting at vector 0:

```
patinfo func1_SRM.pat -vectors 0 8
```

MODULE INFO

Type: SRM
 Name: func1_srm
 Version: 106
 Number of Virtual Vectors: 32
 Number of SRM Chan Locs: 32
 Number of VM Chan Locs: 0
 HasErrors: 0

SRM Chan Data

0	0x00000010	0x66666610	0x00000066
1	0x00000010	0x22222201	0x00000023
2	0x10000010	0x32222200	0x00000022
3	0x01000010	0x23222200	0x00000022
4	0x00100010	0x22322200	0x00000022
5	0x00010010	0x22232200	0x00000022

```
6      0x00001010 0x22223200 0x00000022
7      0x00000110 0x22222300 0x00000022
```

`-ucodes n m` Returns m entries in the VM/SRM microcode table starting from vector n. This switch returns information on a per-module basis. For example, the following command lists the last eight vectors starting at vector 24:

```
patinfo func1_SRM.pat -ucodes 24 8
```

MODULE INFO

Type: SRM
Name: func1_srm
Version: 106
Number of Virtual Vectors: 32
Number of SRM Chan Locs: 32
Number of VM Chan Locs: 0
HasErrors: 0

SRM UCode Data

24	00000000	00000000	00000000	00000000	repeat = 1
25	00000000	00000000	00000000	00000000	repeat = 1
26	00000000	00000000	00000000	00000000	repeat = 1
27	00000000	00000000	00000000	00000000	repeat = 1
28	00000000	00000000	00000000	00000000	repeat = 1
29	00000000	00000000	00000000	00000000	repeat = 1
30	00000000	00000000	00000000	00000000	repeat = 1
31	0x8a000000	00000000	00000000	0x00000060	repeat = 1

`-isl` Returns the Instrument Sync Link (ISL) table. This switch returns information on a per-module basis. For example:

MODULE INFO

Type: SRM
Name: srm_mod_pop_A
Version: 106
Number of Virtual Vectors: 69
Number of SRM Chan Locs: 69
Number of VM Chan Locs: 0
HasErrors: 0

ISL Table:

Temp ICS Code: 0

DCVS:

Pin(s): vcc,

Ucodes: 00000000

Temp ICS Code: 1

DCVS:

Pin(s): vcc,

Ucodes: 0x00000045

Temp ICS Code: 2

DCVS:

Pin(s): vcc,

Ucodes: 0x00000044

Temp ICS Code: 3

DCVS:

Pin(s): vcc,

Ucodes: 0x0000004b

-cwt

Returns the column width table. For example:

Column Width Table:

Column type UCODE width 22

Column type TSET width 9

Column type PIN width 1

Column type PIN width 1

Column type PIN width 8

Column type PIN width 8

-extsymtab

Returns the external symbol table. For example:

Extern Symbol Table:

example_keeplive: type subr, vector 4294967295, offset -1 in VM

-switches

Returns all switch information, including core and platform dependent switches.

It also includes ATPG information. For example:

COMPILER SWITCH TABLE

Core switches:

```

output_filename      default  STRING <pathname>\func.PAT
pinmap_workbook      switch   STRING <pathname>\pinmap.xls
pinmap_sheet         default  STRING
version              default  STRING
opcode_mode          switch   STRING  single
compressed           default  BOOL    1
digital_inst         switch   STRING  HSDM
scan_type            default  STRING
import_all_undefines default  BOOL    0
preprocess           default  BOOL    0
save_comments        switch   BOOL    1
multi_inst           default  BOOL    0
post_processor        default  STRING
post_processor_args   default  STRING
lock                 default  BOOL    0
atpg_translator_id    default  STRING
atpg_translation_timestamp default  STRING
atpg_tool_id          default  STRING
atpg_source_file_name default  STRING
atpg_source_file_id   default  STRING
atpg_source_file_timestamp default  STRING

```

Platform dependent switches:

```

allow_differential    default  BOOL    1
allow_mixed_1x2x      default  BOOL    0
allow_mto_dash        default  BOOL    1
allow_scan_in_srm     default  BOOL    0
cdl_cache             default  BOOL    1
check_set_msb         default  BOOL    1
check_vm_min_size     default  BOOL    1
init_pattern          default  STRING
time_domain           default  STRING
vm_block_size         default  INTEGER 1048576

```

-pik

Returns the primary index key (PIK) assignment table. For example:

PIK Assignment Table:

```

Base Pin Name : a1   chan_offset: 0   chan_type: 3   pik: 256
isGenerated: 0

```

```

Base Pin Name : a2   chan_offset: 0   chan_type: 3   pik: 257
    isGenerated: 0
Base Pin Name : a3   chan_offset: 0   chan_type: 3   pik: 258
    isGenerated: 0
Base Pin Name : a4   chan_offset: 0   chan_type: 3   pik: 259
    isGenerated: 0
Base Pin Name : dir  chan_offset: 0   chan_type: 1   pik: 272
    isGenerated: 0
Base Pin Name : gnd  chan_offset: 0   chan_type: 3   pik: 274
    isGenerated: 0
Base Pin Name : oe   chan_offset: 0   chan_type: 1   pik: 275
    isGenerated: 0
Base Pin Name : vcc  chan_offset: 0   chan_type: 3   pik: 276
    isGenerated: 0
Base Pin Name : ctrl chan_offset: 0   chan_type: 1   pik: 277
    isGenerated: 0
Base Pin Name : dig_pins chan_offset: 0   chan_type: 3   pik: 278
    isGenerated: 0

```

-scanmap

Returns the scan map table. This switch does not return the scan pins used in the pattern. Use -config to determine if the pattern contains scan pins. For example:

Scan Map Table:

```

-----
num_scan_cycles: 200   num_expanded_cycles: 200
virtual_vec_number: 2   start_offset: 2   last_scan_repeat: 0
num_vm_mem_locs: 200

```

-atpgpinmap

Returns the ATPG pin map details contained in the atpg_pinmap construct. For example:

ATE ATPG Pin Map:

```

-----
ATE PinName ATPG PinName
-----
PO0 Block(0).PO{0}
PO1 Block(0).PO{1}
PO2 Block(0).PO{2}
PO3 Block(0).PO{3}
SO0 SO[0]

```

SO1 SO[1]
SO2 SO[2]
SO3 SO[3]

-atpginfo Returns the ATPG information contained in the atpg_info constructs. For example:
ATPG Parameters:

atpg_translator_id: ATEGen Version 1.1.6 [win x86_32] 06-Jun-2010
atpg_translation_timestamp: Sun Oct 24 17:25:01 2010
atpg_tool_id: FastScan v8.2008_4.10-snapshot_2008.10.21_05.00
atpg_source_file_name: ScanTP1.stil
atpg_source_file_id: TP1 with no fails inserted
atpg_source_file_timestamp: Fri Jun 25 09:30:00 2008

ATPG Information:

Name: ScanTP1a_vm
Log first cycle: 0
Log last cycle: 11144
Source ID:

pcpatinfoutility.4.3



Pattern Information Utility : Pattern Analysis Script Example

Pattern Analysis Script Example

This section shows a [Windows batch file \(.bat\)](#) script example used to analyze patterns. To use a Windows batch file script, place it in the same directory that contains the patterns you want to analyze and double-click it to execute. You can also type the batch file name at the command prompt to execute the script.

In this example, the batch file returns the list of pin names and time domains and also provides information that you can use to determine the location of halt opcodes in a list of pattern files. On execution, this script outputs the patinfo results to a text file. You can modify this script to handle your specific pattern management tasks.

Pattern Analysis Batch File

```
for %%f in (*.pat) do patinfo -config %%f >> PatternAuditResults_PinNames.txt
```

```
pause
```

```
findstr /c:"Pattern File:" /c:"Pin Names:" PatternAuditResults_PinNames.txt >  
PatternAuditResults_PinNames_Step2.txt
```

```
pause
```

```
for %%f in (*.pat) do patinfo -hdr %%f >> PatternAuditResults_TD.txt
```

```
pause
```

```
findstr /c:"Pattern File:" /c:"Time domain:" PatternAuditResults_TD.txt >  
PatternAuditResults_TD_Step2.txt
```

pause

for %%f in (*.pat) do patinfo -ucodes 0 10000000 %%f >> PatternAuditResults_halt.txt

pause

		
	2015 Teradyne, Inc. All rights reserved.	

pcpatterncompiler.1.2



[Pattern Compiler](#) : Pattern Compiler Overview

Pattern Compiler Overview

The Pattern Compiler takes an ASCII pattern file that meets the pattern syntax requirements and produces a pattern binary file.

On pattern syntax, see [About Pattern Language](#) in the Pattern Language reference section.

Input and Output Files

The Pattern Compiler expects the input ASCII file to have the extension `.atp` (the extension for an ASCII Teradyne pattern).

The compiler also requires a pin map from an IG-XL test program workbook to validate the names of the pins and pin groups.

The output pattern binary file has the extension `.pat`.

Interfaces

The Pattern Compiler has the following interfaces for specifying filenames and setting various compile-time options:

- | | |
|--|---|
| | <ul style="list-style-type: none">• A graphical user interface. See Pattern Compiler Window. |
| | <ul style="list-style-type: none">• A command-line interface. See Pattern Compiler Command-Line Interface. |
| | <ul style="list-style-type: none">• You can specify most Pattern Compiler options in a pattern file using compiler control statements. If a Pattern Compiler selection conflicts with a compiler control statement, the Pattern Compiler option takes precedence, and the compiler control statement is ignored without generating a warning. |

For more information on the pattern language statements, see [Compiler Control Statements](#).

Pattern Reverse Compiler

IG-XL also provides a [Pattern Reverse Compiler](#), which reads a binary pattern file and recreates the ASCII pattern source file. See [Pattern Reverse Compiler](#).

<		>	
	2015 Teradyne, Inc. All rights reserved.		

pcpatterncompiler.1.3



Pattern Compiler : Pattern Compiler Window

Pattern Compiler Window

The Pattern Compiler window provides controls to facilitate specifying required and optional parameters for pattern compilation. To access this window:

- [From the Start menu](#), select Teradyne IG-XL <version>Tools>Pattern Compiler.
- [From DataTool, on the IG-XL Tools tab](#), in the One-Click Launch group, click Pattern Compiler.

The following topics are available:

- [Interface Overview](#)
- [Pattern Compiler File Specifications](#)
- [Pattern Compiler Options](#)
- [Pattern Compiler Preprocessor Definitions](#)
- [Platform-Specific Options](#)

A command-line interface is also available. See [Pattern Compiler Command-Line Interface](#).

<	>	
	2015 Teradyne, Inc. All rights reserved.	

pcpatterncompiler.1.4



[Pattern Compiler](#) : [Pattern Compiler Window](#) : Interface Overview

Interface Overview

The Pattern Compiler window has the following components:

- Files Frame
- Compiler Options Frame
- Platform-Specific Options Frame
- Additional Controls
- Messages Window

Files Frame

This frame specifies the input and output files:

Pattern Input File	Required. See File Specifications.
Pattern Output File	Optional. See File Specifications.
Pinmap Workbook	Required. See File Specifications.
Pinmap Sheet	Required if the workbook has more than one pin map. See File Specifications.

For more information, see [Pattern Compiler File Specifications](#).

Compiler Options Frame

This frame specifies compiler options. The controls are:

Digital Instrument	See Digital Instrument.
Tester Timing Mode	See Tester Timing Mode.
Scan Type	See Scan Type.
Max Errors	See Max Errors.
Save Comments	See Save Comments.
Import Undefineds	See Import Undefineds.
Pins connected to Multiple Instruments	See Pin Connected to Multiple Instruments.
Lock	See Lock.
Compress	See Compress.
Run Preprocessor	See Run Preprocessor. If you select Run Preprocessor, the Preprocessor Definitions button is enabled
Preprocessor Definitions	See Pattern Compiler Preprocessor Definitions.
Run Customer's Post-Processor	See Run Customer's Post-Processor.

Platform-Specific Options Frame

In addition to the Compiler Options, there are Platform-Specific Options. The options available depend on the value selected for Digital Instrument.

See Platform-Specific Options.

Additional Controls

The Pattern Compiler window has the following additional controls:

Compile	Starts compilation.
Exit	Exits the Pattern Compiler window.
Help	Brings up help for the Pattern Compiler.
About	Displays the compiler version number.

Messages Window

Any compilation errors are displayed in the Messages window. The current status of compilation is displayed in the small window beneath the Messages window.

< >		
	2015 Teradyne, Inc. All rights reserved.	

pcpatterncompiler.1.5

<

>

[Pattern Compiler](#) : [Pattern Compiler Window](#) : [Pattern Compiler File Specifications](#)

Pattern Compiler File Specifications

File Specifications

The Files frame of the pattern compiler interface lets you specify files for input, output, and the pin map. The frame contains Browse buttons that let you navigate to a file. The controls in this frame include:

Pattern Input File	Specifies the ASCII input file, which has the .atp extension. You must specify an input file.
Pattern Output File	Specifies the output filename. Note that the characters "+" (plus sign) and "," (comma) are invalid both in the pattern file filename and in the pathname to the pattern file. See Pattern File Filename and Pathname in the Pattern Language section.
Pinmap Workbook	Specifies the IG-XL test program workbook containing the pin map needed to resolve the pin and pin group names used in the pattern file. A workbook has an .xls extension.
Pinmap Sheet	Specifies a Pin Map sheet in the pin map workbook. The sheet name is the name that appears on the tab for this sheet. If a workbook contains only one Pin Map sheet, you can omit this parameter. If a workbook contains more than one Pin Map sheet, you must specify a sheet name.

Compiling Multiple Files

To compile multiple files:

1. [Click the](#) Browse button to bring up the Open dialog box.

1. [Hold down the](#) Ctrl key and left-click the files you want.

2. [Click](#) Open.

You can also use the standard wildcard characters * and ? to match a set of input files.

[When you select multiple files:](#)

- [The](#) Pattern Input File box displays the message <multiple files from dialog box>.

- [The](#) Pattern Output File box and the Browse button are grayed out. You cannot specify an output file name. IG-XL uses the default output filenames for the multiple files.

- [All input files must use the same workbook and, if specified, the same Pin Map sheet.](#)

- [The pattern compiler GUI has a limit of 256 files. If the wildcard template you enter matches more than 256 input files, the pattern compiler returns an error and does not compile any files.](#)



	2015 Teradyne, Inc. All rights reserved.	
--	--	--

pcpatterncompiler.1.6

<	>
-----------------	-----------------

[Pattern Compiler](#) : [Pattern Compiler Window](#) : Pattern Compiler Options

Pattern Compiler Options

The Compiler Options section of the interface lets you select specific compiler options. This section also contains a section labeled Platform-specific Options. See [Platform-Specific Options](#). Compiler Options are:

- | | |
|---|---|
| • | Digital Instrument |
| • | Tester Timing Mode |
| • | Scan Type |
| • | Max Errors |
| • | Save Comments |
| • | Import Undefineds |
| • | Pin Connected to Multiple Instruments |
| • | Lock |
| • | Compress |

- [Run Preprocessor](#)
- [Run Customer's Post-Processor](#)

Digital Instrument

The digital instrument for which this pattern is to be compiled. The selection here determines whether certain controls are enabled and what values are available for other controls. It also brings up platform-specific options for this instrument. See [Platform-Specific Options](#).
For UltraFLEX, the digital instrument options are:

HSD4G	UltraPin4000 and HPM
HSDM	HSD1000 and UltraPin800
HSDMQ	UltraPin1600
Serial6G	SB6G
Serial10G	UltraSerial10G

Tester Timing Mode

Select the timing mode for which the pattern should be compiled. The options are:

HSD4G	• by1 • by1_tof (HPM only) • by16 • by16iio (UltraPin4000 only)
HSDM	• single • dual

HSDMQ

- single
- dual
- quad

Scan Type

Specifies the method for packing scan data into memory. The options are:

x3, x6, x12, x24	These scan types specify scan packing methods that allow 2-bit scan chains. The number after x specifies packing efficiency: data of 3/6/12/24 scan cycles is packed into one parallel vector.
x2, x4, x8	These scan types specify scan packing methods allow three-bit scan chains. The number after x specifies packing efficiency: data of 2/4/8 scan cycles is packed into one parallel vector.
extended	This scan type allows three-bit scan chains. Data of 12 scan cycles is packed into one parallel vector.
parallel	This scan type allows expanding scan vectors into parallel vectors. Data of one scan cycle is packed into one parallel vector. Scan data is converted to parallel data. This type allows either two-bit or three-bit scan chains. No compression is achieved with this scan type. Vector memory is consumed quickly.

For more information, see [Selecting SCAN Type](#).

Max Errors

Specifies the number of errors that will cause compilation of the pattern file to be aborted. The default is 200.

A value of 0 means that compilation is never aborted because of too many errors.

Save Comments

If checked, saves comments in the pattern binary, for use by the PatternTool and the Pattern Reverse Compiler. By default the box is checked.

If not checked, any comments in the ASCII pattern file are stripped when the pattern binary is created. This saves disk space.

This check box overrides any `save_comments` statement in the `.atp` file. See [Saving Comments in a Compiled File](#) in the pattern language reference.

Import Undefineds

If checked, imports any undefined subroutines labels and tset names without giving an error or warning.

If not checked, raises an error if any label or tset name is undefined at the end of compilation. This is the default.

This option is for use by certain post-processors that generate names on the fly. Avoid using this option. SRM usage cannot be minimized and errors in names are not detected until pattern load time.

Pin Connected to Multiple Instruments

If checked, indicates to the compiler that a single pin may be connected to multiple instruments on the channel map.

If not checked, each pin is connected to no more than one instrument. This is the default.

The effect of checking this box is to suppress the compiler's error messages about duplicate pins. If this box is not checked, and if any pins are connected to multiple instruments, the duplicate pins error message is raised. If you choose to suppress the error message by checking this box, make sure that you have no unintended duplicate pins. The compiler cannot perform this check.

You must select this option when you are using a digital channel as secondary instrument through DIB Access: that is, when a pin type is set to Analog in the pin map, but appears in the vector statement of the pattern file.

Lock

If checked, locks the compiled pattern file.

If not checked, does not lock the compiled pattern file. This is the default.

A locked pattern file cannot be opened by PatternTool or HRAM Display. It also cannot be reverse-compiled. See [Pattern Locking](#).

Compress

If checked, compresses the compiled pattern file. This is the default.

If not checked, does not compress the compiled pattern file.

If you select compression, the Pattern Compiler does not compress the entire file. Instead, it compresses pattern data internally, so that individual blocks of data within the pattern file are compressed. The compressed file as a whole still has the extension `.pat`.

Compressing a pattern file affects the performance of PatternTool, especially with larger patterns, although you may not notice the effect. PatternTool may take a few seconds longer to load and to save a file, and scrolling through the file may not always seem smooth.

Run Preprocessor

If checked, runs the C++ preprocessor on this file.

If not checked, the pattern compiler does not run the C++ preprocessor. This is the default.

You must select this option if the patterns use preprocessor constructs such as `#include` or `#define`. If you omit this option and the pattern compiler recognizes a preprocessor construct, compilation is aborted, and you are asked if the pattern compiler should be rerun with this option. If you select this option, the Preprocessor Definitions button is enabled. See [Pattern Compiler Preprocessor Definitions](#).

Run Customer’s Post-Processor

Click this button to bring up a window that specifies a post-processor to be run on the input `.atp` file, plus any arguments to the post-processor.

For more information, see [post_processor](#) in the Pattern Language section.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

pcpatterncompiler.1.7

	
---	---

[Pattern Compiler](#) : [Pattern Compiler Window](#) : Pattern Compiler Preprocessor Definitions

Pattern Compiler Preprocessor Definitions

If you select [Run Preprocessor](#), the [Preprocessor Definitions](#) button is enabled. This button lets you define the values of macros to be passed to the C++ preprocessor. These definitions are equivalent to the `#define` statement in a C++ program source file.

When you click the [Preprocessor Definitions](#) button, a dialog box appears. You enter each definition on a separate line. When you finish the definitions, click OK.

Formats

The two formats for definitions are:

name =value	Replaces every occurrence of name with value in the pattern file. Spaces are not permitted around the =. For example: MAX_ROWS = 500
name	Defines name as the number one (1). Used for conditional compiling with <code>#ifdef</code> statements.

		
	2015 Teradyne, Inc. All rights reserved.	

pcpatterncompiler.1.8

<

>

Pattern Compiler : [Pattern Compiler Window](#) : Platform-Specific Options

Platform-Specific Options

In addition to the compiler options, there are platform-specific options. The options available depend on the value selected for Digital Instrument. For each option:

1. [Use the](#) Option drop-down list to select the option to set.
2. [Use the](#) Value drop-down list to select the value you want for the option.
3. [Click](#) OK to set this selection.

Available digital instruments are:

- [HSDM, HSDMQ, and HSD4G Options](#)
- [Serial6G Options](#)
- [Serial10G Options](#)

HSDM, HSDMQ, and HSD4G Options

For the HSDM, HSDMQ, and HSD4G digital instruments, the options are:

cdl_cache	Select yes or no. CDL caching is hardware technique for compressing vector data. It uses CDL (Channel Data Latch) structure and a small hardware cache of vector
---------------------------	---

[data.](#)

- [yes](#): Turns on CDL caching to produce a smaller binary pattern file. Default.
- [no](#): Turns off CDL caching to reduce compilation time.

[init_pattern](#)

Specify the pattern file, pattern module, or pattern set to be executed before this pattern file is executed. Values for [init_pattern](#) can be:

- ["pathname\patternfile.pat\[:patternmodule\]"](#)
- ["patternmodule"](#)
- ["patternset"](#)

If the initialization pattern is another pattern file, you must specify the pathname. You can specify a pattern module within the pattern file. If the pattern module is in the current pattern file, you do not need to include the pattern file name.

You can also specify a pattern set name.

See [Initialization Pattern](#) in the Pattern Language section.

[check_set_msb](#)

Select [yes](#) or [no](#).

The [set](#) or [set_infinite](#) opcode must be preceded by at least one [set_msb](#) or [set_msb_infinite](#) opcode in a pattern module. This control allows you to turn on or off the check for this requirement during compilation. The default is turning on this check.

[time_domain](#)

Specify the time domain to be used for pins in this pattern.

If the pins specified in the pattern file do not match any of the specified time domains, then the pattern compiler sets the time domain to the "All" domain, which is a single time domain that is comprised of all pins defined in the test program.

You can also use one of these keywords, instead of the time domain name:

- [tlSystem](#): Assign a pattern to the default System time domain in a multiple time domain pin map.
- [tlBestFit](#): Assign a pattern to the best fit time domain in a multiple time domain pin map. This option allows you to override a non-default time domain name in the ATP file.

See [time_domain](#) in the Pattern Language section.

[allow_mto_dash](#)

Select yes or no.

In MTO patterns, setting this option to yes lets you select that default pin setup that supports the pin state "-" to indicate runtime repeat.

The default is no, not to support use of the dash.

This option is not used on the UltraPin4000 and HPM instruments.

Serial6G Options

For the Serial6G digital instrument, the options are:

[check_vm_min_size](#)

Select yes or no.

Whether to check the minimum number of vectors within each module.

If this option is enabled (yes), the vector size must be ≥ 1024 ; otherwise, an error message is returned.

The default is yes, to perform the check.

[check_vm_mod_size](#)

Select yes or no.

Whether to check that the total number of vectors within each module is a multiple of 1024. If this option is enabled (yes), the number of vectors should be a multiple of 1024; otherwise, an error message is returned.

The default is yes, to perform the check.

Serial10G Options

For the Serial10G digital instrument, the options are:

[check_oob_size](#)

Select yes or no.

Whether to check that total number of out of band (OOB) regions within each module is a multiple of 10. If this option is enabled (yes), each OOB region must start and end on a 10-bit symbol boundary; otherwise, an error message is returned.

check_vm_mod_size

The default is no, to disable the check.

Select yes or no.

Whether to check that the total number of vectors within each module is either a multiple of 2560, or in the case of a pattern set with only one pattern or the last module in a pattern set, a multiple of 40. If this option is enabled (yes), the number of vectors should be a multiple of 2560 or 40; otherwise, an error message is returned.

The default is no, to disable the check.

<		>	
	2015 Teradyne, Inc. All rights reserved.		

pcpatterncompiler.1.9

<

>

Pattern Compiler : Pattern Compiler Command-Line Interface

Pattern Compiler Command-Line Interface

To start the Pattern Compiler from the command prompt, select Start>Programs>Accessories>Command Prompt.

The command-line interface to the pattern compiler is:

apc input-ascii-file(s) [switches]

Input Files

The input file name is required, unless you use the -stdin switch to specify standard input. The expected extension is .atp, or .atp.gz for a compressed file.

You can specify more than one input file. A separate pattern binary output file is created for each input file. If you specify more than one file, accept the default output filename, instead of using the -output switch.

You can use the standard wildcard characters * and ? in specifying the input filenames.

For more information, see Using stdin.

Switches

The pattern compiler switches are listed here. You can prefix a switch word with "/" as well as with "-". The two are equivalent. For more information, see the comparable item in Pattern Compiler Window.

-output filename	Specifies the output filename.The default is the input filename plus the .pat extension.
-pinmap_workbook filename	Specifies the workbook that contains the pin map information needed to resolve the pin and pin group names used in the pattern file.

<code>-pinmap_sheet sheetname</code>	Specifies a Pin Map sheet in the pin map workbook. If the workbook contains only one Pin Map sheet, you can omit this parameter.
<code>-digital_inst inst</code>	Specifies the digital instrument. This enables instrument-specific options. The options are HSD4G, HSDM, HSDMQ, and Serial6G. The instrument-specific options are listed below.
<code>-opcode_mode mode</code>	Forces opcode mode to be mode. For HSDM mode is single or dual. HSDMQ mode is single, dual, or quad. For HSD4G mode is by1 , by1_tof (HPM only), by16, or by16iio (UltraPin4000 only).
<code>-version</code>	Version of pattern file(s) to compile.
<code>-scan_type scantype</code>	Type of scan data.
<code>-import_all_undefineds</code>	Automatically import all undefined symbols.
<code>-cpp</code>	Runs the C++ preprocessor on this file. By default, the pattern compiler does not run the C++ preprocessor.
<code>-comments</code>	Saves comments in the pattern binary for use by the pattern tools and the Pattern Reverse Compiler. If you omit this switch, the pattern compiler uses the setting in the source .atp file. By default, the .atp file does not specify saving comments. See Saving Comments in a Compiled File in the pattern language reference.
<code>-max_errors n</code>	Specifies the number of errors that will cause compilation of the pattern file to be aborted. The default is 200. A value of 0 means never to abort compilation because of too many errors.
<code>-nocompress</code>	Does not compress pattern data blocks in the pattern file.
<code>-logfile filename</code>	Directs any compiler messages to the specified filename. By default, messages are logged to input-file.log.
<code>-suppress_log</code>	Disables output to the main log file.
<code>-multiinst</code>	Indicates that more than one instrument is connected to a single pin.
<code>-define name[=value]</code>	Defines the values of macros to be passed to the C++ preprocessor. The formats for definitions are: <code>name=value</code>: Replaces every occurrence of name with value in the pattern file. There can be no spaces around

the =.

- **name:** Defines <name> as the number one (1). Used for conditional compiling with `#ifdef` statements.

During compilation, the `-define` keyword can be used only once per individual pattern, but it can take a space-delimited list as a argument. For example:

```
-define INSERT2 MYDEF=XXX Z=3
```

<code>-i paths</code>	Includes paths to be passed to the C-preprocessor.
<code>-post-processor filename</code>	Path name of customer's postprocessor executable.
<code>-post-processor_args args</code>	Arguments of customer's postprocessor.
<code>-lock</code>	Prevents pattern from being reverse compiled or opened in PatternTool. See Pattern Locking.
<code>-help, -?</code>	Prints a summary of how to invoke the pattern compiler and what switches are valid.
<code>-stdin</code>	Compiles data from standard text data input, instead of a .atp file. See Using stdin.

Digital Instrument Switches

The following switches can be used when `-digital_inst` specifies HSDM, HSDMQ, or HSD4G. See Platform-Specific Options.

<code>-cdl_cache yes/no</code>	Turns on/off CDL caching. The default is yes.
<code>-init_pattern init-pattern</code>	Uses the specified pattern module, pattern file, or pattern set as an initialization pattern.
<code>-check_set_msb yes/no</code>	Turns on/off the check to make sure the set or set_infinite opcode is preceded by a set_msb/set_msb_infinite opcode. The default is yes.
<code>-time_domain domain-name tlSystem tlBestFit</code>	Specifies the time domain for pins in the compiled pattern. If the pins specified in the pattern file do not match any of the specified time domains, then the pattern compiler sets the time domain to the "All" domain, which is a single time domain that is comprised of all of the pins defined in the test program.

You can also use one of these keywords, instead of the time domain name:

- **tlSystem**: Assign a pattern to the default System time domain in a multiple time domain pin map.
- **tlBestFit**: Assign a pattern to the best fit time domain in a multiple time domain pin map. This option allows you to override a non-default time domain name in the ATP file.

See **time_domain** in the Pattern Language section.

-allow_mto_dash yes/no

Turns on/off the ability to use "-" to indicate runtime repeat in MTO patterns. The default is no.

This option is not used on the UltraPin4000 and HPM instruments.

The following switches can be used when **-digital_inst** specifies Serial6G. See **Platform-Specific Options**.

-check_vm_min_size yes/no

Turns on or off the check that the vector size of each module is ≥ 1024 . The default is yes.

-check_vm_mod_size yes/no

Turns on or off the check that the number of vectors in each module is a multiple of 1024. The default is yes.

The following switches can be used when **-digital_inst** specifies Serial10G. See **Platform-Specific Options**.

-check_oob_size yes/no

Turns on or off the total number of out of band (OOB) regions within each module is a multiple of 10. The default is no.

-check_vm_mod_size yes/no

Turns on or off the check that the number of vectors in each module is a multiple of 2560, or in the case of a pattern set with only one pattern or the last module in a pattern set, a multiple of 40. The default is no.

Using stdin

The `switch -stdin` compiles data from standard text data input, instead of an ASCII Teradyne pattern file (.atp). It takes the following format:

```
<standard text data input> | apc -stdin <output pattern filename>.pat
```

Do not use the `-cpp` switch or specify any .atp files when using this option. If you do not specify an output pattern file name using the `-output` switch, the pattern compiler writes the binary file to the default filename, `apc_stdin.pat`.

This switch is useful as an alternative to generating large .atp files. For instance, you can pipe standard text data from a vector generator script directly into the compiler, rather than creating an .atp file first and then compiling that file:

```
myVectorGeneratorScript | apc -stdin -output myOutputFile.pat
```

The following example executes the pattern compiler as a child process and directs the output from this parent process to the child's standard input:

```
#include <stdio.h>
```

```
#include <stdlib.h>
```

```
void main(void)
```

```
{
```

```
    char buffer[1024];
```

```
    FILE* chkdsk;
```

```
    // Execute APC compiler as a child process and direct output from
```

```
    // this parent process to the child's stdin.
```

```
    if((chkdsk = _popen( "apc -stdin -output bar.pat", "wt" )) == NULL )
```

```
        exit( 1 );
```

```
    FILE* patFile = fopen("foo.atp", "r");
```

```
    while((fgets(buffer, 1024, patFile) != 0)) {
```

```
        fprintf(chkdsk, "%s", buffer);
```

```
    }
```

```
    fclose(patFile);
```

```
    /* Close pipe and print return value of CHKDSK. */
```

```
    printf( "\nProcess returned %d\n", _pclose( chkdsk ) );
```

```
}
```

<	>	
	2015 Teradyne, Inc. All rights reserved.	

pcpatternlock



Pattern Locking

Pattern Locking

IG-XL supports pattern locking. This lets you protect a compiled .PAT binary pattern file. Locking a pattern file prevents further viewing or modification by PatternTool or HRAM Display. It also prevents reverse compiling.

You cannot unlock a locked pattern file. You must recompile it without setting the locked option.

Locking at Compilation

You can lock a pattern by compiling it using the Lock option, in one of these ways:

- The Pattern Compiler GUI has a Lock check box.
- The command-line apc interface has a -lock switch.
- You can include the compiler control statement `lock = yes;` in the .atp file.

For more information, see:

- Pattern Compiler Window
- Pattern Compiler Command-Line Interface
- General Compiler Control Statements

PatLock Utility

You can also use the PatLock utility to lock a pattern that has already been compiled. The command is:

> patlock input-files

The input files are the pattern files to be locked. The filename must match exactly, including the .pat or .pat.gz extension. You can use wildcard characters for a template match. Multiple filenames are separated by spaces.

☒ **Note:**

You cannot lock a pattern file that was compiled under versions of IG-XL earlier than V7.20.01_uflx, when pattern locking was introduced. To lock such a file, you must first recompile it under V7.20.01_uflx or later.

Locking Pattern Sets

To lock any pattern file in a pattern set, you must lock all patterns in the pattern set. If only some patterns in the set are locked, a tool may be able to open any of the pattern files, including the locked ones.

<

>

2015 Teradyne, Inc. All rights reserved.

pcpatternreverse.2.01

<	>
-----------------	-----------------

Pattern Reverse Compiler

Pattern Reverse Compiler

The Pattern Reverse Compiler creates an ASCII pattern from a compiled binary pattern file (extension .pat).

The following topics are available:

- | | |
|---|---|
| • | Pattern Reverse Compiler Overview |
| • | Pattern Reverse Compiler Window |
| • | Pattern Reverse Compiler Command-Line Interface |
| • | Notes on the Pattern Reverse Compiler |

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

pcpatternreverse.2.02



[Pattern Reverse Compiler](#) : Pattern Reverse Compiler Overview

[Pattern Reverse Compiler Overview](#)

The Pattern Reverse Compiler creates an ASCII pattern from a compiled binary pattern file (extension .pat).

For specifics of what reverse compilation does, see the [Notes on the Pattern Reverse Compiler. Input and Output Files](#)

The Pattern Reverse Compiler expects the input pattern binary file to have the extension .pat (the extension for a binary Teradyne pattern).

The output ASCII file has the extension .atp.

[Interfaces](#)

The pattern reverse compiler has the following interfaces for specifying filenames and setting various compile-time options:

- [A graphical user interface. See Pattern Reverse Compiler Window.](#)
- [A command-line interface. See Pattern Reverse Compiler Command-Line Interface.](#)

[Pattern Compiler](#)

IG-XL also provides a Pattern Compiler, which reads an ASCII pattern source file and produces a binary pattern file. See [Pattern Compiler](#).



	2015 Teradyne, Inc. All rights reserved.	
--	--	--

pcpatternreverse.2.03



Pattern Reverse Compiler : Pattern Reverse Compiler Window

Pattern Reverse Compiler Window

The Pattern Reverse Compiler window provides controls to facilitate specifying required and optional parameters for reverse compilation.

To access the Pattern Reverse Compiler window, from the Start menu select Teradyne IG-XL version>Tools>Pattern Reverse Compiler.

The following topics are available:

- [Interface Overview](#)
- [Executing Reverse Compilation](#)
- [Reverse-Compiling Multiple Files](#)
- [Pattern Reverse Compiler Column Width](#)
- [Pattern Reverse Compiler Include Files](#)

A command-line interface is also available. See [Pattern Reverse Compiler Command-Line Interface](#).



	2015 Teradyne, Inc. All rights reserved.	
--	--	--

pcpatternreverse.2.04

<

>

[Pattern Reverse Compiler](#) : [Pattern Reverse Compiler Window](#) : Interface Overview

Interface Overview

The Pattern Reverse Compiler window has the following components:

- Files Frame
- Compiler Options Frame
- Spacing Options Frame
- Additional Controls
- Messages Window

Files Frame

This section specifies the input and output files:

Pattern Input File:	Required.The compiled pattern file to be uncompiled. Use the Browse button to specify a file.The pattern file can be uncompressed (.pat) or compressed (.pat.gz).
Pattern Output File:	Optional. The output file. If you do not specify an output file, the default is the input filename with the extension .atp. If the file already exists, you are asked to confirm overwriting it, unless you have checked the Force box. The output file can be uncompressed (.atp) or compressed (.atp.gz).

All Include File(s): Optional. The read-only menu displays the filenames for which `#include` statements are to be generated. To create or modify the list, use the Include Files button under Compiler Options.

Compiler Options Frame

Compiler options control certain actions of the reverse compiler:

Force	Overwrites any existing output file without confirmation.
Include Files	Check this box to enable the button that will bring up a dialog box to create or modify the list of filenames for which <code>#include</code> statements are to be generated. See the description of the Include Files dialog box under Pattern Reverse Compiler Include Files .
Vector Numbers	Adds vector number as comments in the output file.
Compress	Compresses the output file, which will have the extension <code>.atp.gz</code> .

Spacing Options Frame

Spacing options control the appearance of the reverse compiled data in the output file:

No Alignment	Do not try to align the output data in columns. Columns are separated by a single space. If you check No Alignment, the Column Widths option is disabled. See Column Alignment .
Page Width	The maximum number of characters per line on the output page. By default, all data for a single vector is put on one line, unless a second line is needed for <code>mux</code>, <code>io_midband</code> or <code>io_valid</code> data. This option overrides the default behavior if the data would be longer than the specified width. This switch does not affect comments, which will remain their original width. If you specify a Page Width, the No Alignment and Column Widths options are disabled.
Page Length	Length of pages in the output file, in lines. The value 0 means not to break the output into pages. The default is 0.
Column Widths	Overrides the width to be used for specific columns in the output. Use the Set button to bring up a dialog box for entering column widths. See the description of Pattern Reverse Compiler Column Width for details about setting the column widths.

Additional Controls

The Pattern Compiler window has the following additional controls:

Uncompile	Starts revers compilation.
Exit	Exits the Pattern Reverse Compiler window.
Help	Brings up help for the Pattern Reverse Compiler.
About	Displays the reverse compiler version number.

Messages Window

Any compilation errors are displayed in the Messages window.The current status of compilation is displayed in the small window beneath the Messages window.

<

>

2015 Teradyne, Inc. All rights reserved.

pcpatternreverse.2.05

<	>
-----------------	-----------------

Pattern Reverse Compiler : [Pattern Reverse Compiler Window](#) : Executing Reverse Compilation

Executing Reverse Compilation

When you have specified files and chosen any compiler options, [click the](#) Uncompile button. Any compilation errors are displayed in the [Messages window](#).

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

pcpatternreverse.2.06

<	>
-----------------	-----------------

Pattern Reverse Compiler : [Pattern Reverse Compiler Window](#) : Reverse-Compiling Multiple Files

Reverse-Compiling Multiple Files

To compile multiple files:

1. [Click the](#) Browse button to bring up the Open dialog box.

1. [Hold down the](#) Ctrl key and left-click the files you want.

2. [Click](#) Open.

When you select multiple files:

- [The](#) Pattern Input File box displays the message <Multiple files are selected>.
- [Do not specify an output file name. Instead, use the default output filenames for the multiple files.](#)

<	>
-----------------	-----------------

pcpatternreverse.2.07

	
---	---

[Pattern Reverse Compiler](#) : [Pattern Reverse Compiler Window](#) : Pattern Reverse Compiler Column Width

Pattern Reverse Compiler Column Width

Nondefault Column Width

This option overrides the width used for specific columns in the output. For example, "2=10" says that the second setting is to be 10 characters wide.

In numbering the columns, column 1 is the pattern microcode column. For example, "1=12" means that the microcode column should be 12 characters wide. The setting "2=10" refers to the first column of data.

If column text is wider than the specified width, it is not truncated. Instead, it uses as many characters as necessary, and the rest of the line is shifted left. As a result, the remaining columns in the vector row will not be aligned.

Using the Column Widths Box

To enter a column setting on the dialog box, enter the column number in **Line** and the number of characters in **Width**. Then click **Add**. The setting is added to the **Output display area**, in the format **column=width**. For example: 2=10.

To modify or delete a width equation already on the menu, select the equation from the list, and then click **Modify** or **Delete**.

To clear all equations, click **Delete All**.

When you finish entering column settings, click **Done**. The settings appear in the **Column Widths display area** of the main **Pattern Reverse Compiler** dialog box.

	
---	---

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

pcpatternreverse.2.08

<	>
-----------------	-----------------

Pattern Reverse Compiler : Pattern Reverse Compiler Window : Pattern Reverse Compiler Include Files

Pattern Reverse Compiler Include Files Include Files

This option generates an #include statement for each filename you specify. When you initially compile a pattern file, the C++ preprocessor removes any #include statements. To recompile the reverse-compiled output, you must explicitly identify files to be included.

Using the Include File Box

To enter an include file, enter the filename in File Name. Then click Add. The filename is added to the All Include Files menu.

To modify or delete a file already on the menu, select the file from the list, and then click Modify or Delete.

To clear all files, click Delete All.

When you finish entering filenames, click Done. The filenames appear in the All Include File(s) menu on the main Pattern Reverse Compiler dialog box.

<	>
-----------------	-----------------

	2015 Teradyne, Inc. All rights reserved.	
--	--	--

pcpatternreverse.2.09

<

>

Pattern Reverse Compiler : Pattern Reverse Compiler Command-Line Interface

Pattern Reverse Compiler Command-Line Interface

The pattern reverse compiler creates an ASCII pattern file from a compiled binary pattern file (extension .pat).

For specifics of reverse compilation, see [Notes on the Pattern Reverse Compiler](#).

Invoking from the Command Line

Invoke the pattern reverse compiler from the command line as follows:

`aprc input-binary-file [switches]`

Input File

The input file name is required. The expected extension is .pat. The extension .pat.gz indicates a compressed input file.

Switches

The pattern reverse compiler switches are listed here. You can prefix a switch word with "/" as well as with "-". The two are equivalent.

-output filename	Specifies the name to give to the output file. The default is the input filename with a .atp extension. The extension .atp.gz indicates a compressed file.
-force	Overwrites any existing output file without confirmation.
-include filename(s)	Generates #include statements for the specified filenames. Names must be separated by white space.
-noalign	Tells the reverse compiler not to try to align the output data in columns. See Column Alignment .
-vectnum	Adds vector numbers as comments in the output file.

-width n	Specifies the maximum width of lines in the output file. By default, the pattern reverse compiler puts all data for a single vector on one line, unless a second line is needed for 2x data. This switch overrides the default behavior if the data would be longer than the specified width. This switch does not affect comments, which will remain their original width.
-length n	Specifies the length of pages in the output file. The value 0 means not to break the output into pages, and is the default.
-column_width n1=w1	Overrides the width used for specific columns in the output. For instance: -column_width 1=30 2=10 5=20 would force the pattern microcode column (always column 1) to be 30 characters wide, column 2 to be 10 characters wide, and column 5 to be 20 characters wide. Note that text is not truncated if it is too big for the specified width. It is output anyway and subsequent data for that vector may not be aligned neatly.
-compress	Compresses the output file.
-help, -?	Prints a summary of how to invoke the pattern reverse compiler and which switches are valid.

< >	
	2015 Teradyne, Inc. All rights reserved.

pcpatternreverse.2.10



[Pattern Reverse Compiler](#) : Notes on the Pattern Reverse Compiler

Notes on the Pattern Reverse Compiler

The pattern reverse compiler generates an ASCII source file from a compiled binary pattern file. It is not guaranteed to recreate exactly the same ASCII as the original ASCII input file, because of how the pattern compiler handles some of the input file. Some details are given below.

If the output of the pattern reverse compiler is compiled and then again reverse-compiled, the resulting output should match the original output, assuming no change in the switches used.

The following topics are available:

- [Compiler Control Statements](#)
- [C++ Preprocessor](#)
- [Column Alignment](#)
- [Comments](#)
- [Tset Data](#)
- [Case of Data](#)
- [Scan Data](#)
- [Obsolete Pattern Language Names](#)

- [MTO Opcodes](#)

Compiler Control Statements

The pattern reverse compiler generates compiler control statements for options that were specified by control statement in the original input file. It does not generate these statements for options specified only at compile-time, nor for control statements that were overridden by compile-time options.

C++ Preprocessor

Information removed from the original ASCII source file by the C++ preprocessor (`#ifdef`, `#include`, plus any macros before expansion) is not available to the reverse compiler and will not appear in the reversed-compiled output.

Column Alignment

By default, the pattern reverse compiler attempts to align ASCII pattern data in columns for readability. Because the column width is based on the width of the user's input data, a column may occasionally be wider than necessary. For example, the pattern reverse compiler may produce `.X` (indicating a repeated character) when your original data was `XXXXXXXX`.

To override the default output format, check the `No Alignment` box under `Spacing Options`.

Comments

The pattern compiler strips out comments, unless a compile-time option specifies otherwise. If comments are retained, the pattern compiler may move and consolidate them within the binary file. For this reason, the ASCII output of the pattern reverse compiler may differ from the original ASCII input.

Tset Data

In the vector statement, the pattern reverse compiler produces a tset column only if the original pattern had a tset column; it is in the same place within the pinlist as the original tset column. If time sets are specified only on selected vectors, the pattern reverse compiler produces a time set on every vector.

Case of Data

For the data of a vector statement, the pattern reverse compiler produces uppercase tester data characters (unless you redefined them) and uppercase hex numbers.

Scan Data

For scan data, the pattern reverse compiler replaces data for scan pins in a `scan_setup` statement with data from the first cycle of the next scan vector.

Obsolete Pattern Language Names

The names used in the IG-XL pattern language have been changed, based on user experience. These changes are designed to make the name indicate more clearly the function of the opcode, control bit, or flag.

To maintain compatibility with existing pattern files, the pattern compiler accepts both the new names and the obsolete names. The pattern reverse compiler, however, generates only the new names.

MTO Opcodes

Every MTO opcode is programmed on every vector. If the source ASCII file does not explicitly program an MTO opcode on a vector, a default is used.

To maintain readability, the pattern reverse compiler prints only those MTO opcodes that have nondefault values. Even if the source ASCII file included an MTO opcode with a default value, the reverse-compiled output does not show the default.

		
	2015 Teradyne, Inc. All rights reserved.	